

Projet Operations Research

Implémentation d'algorithmes de flots

STEFANI Romain

9 mai 2026

1 Introduction

Ce projet implémente différents algorithmes de flots dans les graphes : Ford-Fulkerson pour le calcul du flot maximum et de la coupe minimale, deux variantes du min cost flow par chemins augmentants successifs (Bellman-Ford et Dijkstra avec renormalisation), et un algorithme de détection de cycles négatifs.

2 Structure de données

Pour la structure de données, on va avoir besoin de 3 types, les arcs, les sommets et le réseau. On va en rajouter un dernier, le **ResultatFlotMax**, qui nous permet de renvoyer un résultat et de l'afficher proprement.

Tous nos algorithmes vont manipuler un graphe résiduel, c'est-à-dire un graphe contenant les arêtes du réseau mais également des arêtes allant dans le sens opposé à chaque arête du graphe. Ce sera la principale difficulté pour créer nos données.

Pour nos 3 types, on a les sommets qui ont un nom et une liste d'arcs sortants, le réseau qui a une source, un puits, une liste de sommets et une liste d'arcs, et les arcs qui ont une capacité, une valeur de flot et un coût. Les arcs ont aussi un sommet source et un sommet destination puisqu'ils sont orientés, et pour pouvoir manipuler le graphe résiduel facilement on leur ajoute l'arc inverse ainsi qu'un booléen **estInverse** pour savoir s'il s'agit de l'arc original ou de l'inverse.

L'ajout du pointeur vers l'arc inverse permet de facilement déplacer du flot car quand on ajoute du flot sur un arc on doit enlever la même quantité dans l'arc inverse et donc grâce au pointeur on n'a pas besoin de reparcourir la liste des arcs pour savoir lequel modifier.

3 Les algorithmes

3.1 Ford-Fulkerson et coupe minimale

Le principe de Ford-Fulkerson est simple, tant qu'il existe un chemin de la source au puits dans le graphe résiduel, on l'utilise pour ajouter du flot, et on s'arrête quand il n'y en a plus. Pour trouver ces chemins, on fait un BFS dans le graphe résiduel, ce qui revient à parcourir les arcs sortants de chaque sommet en filtrant sur la capacité résiduelle. Pendant le BFS, on retient pour chaque sommet l'arc qui a permis de l'atteindre, ce qui permet de remonter le chemin une fois qu'on arrive au puits.

Une fois le chemin trouvé, on calcule la capacité résiduelle minimum sur le chemin, c'est notre delta, puis on l'ajoute en flot sur chaque arc du chemin. Comme on a le pointeur vers l'arc inverse, l'ajout met aussi à jour l'inverse automatiquement. On boucle jusqu'à ce que le BFS ne trouve plus de chemin, et la somme des delta donne le flot maximum.

Pour trouver la coupe minimale, on regarde après l'algorithme dans quel état est le résiduel. On part de la source et on note tous les sommets qu'on peut encore visiter, ils forment un groupe S . Tous les autres sommets, dont le puits, forment un groupe T . La coupe minimale est l'ensemble des arcs originaux qui partent d'un sommet de S et arrivent sur un sommet de T .

3.2 Min cost flow avec Bellman-Ford

Pour le min cost flow, on cherche le flot maximum mais cette fois en minimisant le coût total. Le principe est exactement le même que Ford-Fulkerson, on remplace juste le BFS par une recherche du chemin de coût minimum dans le résiduel.

Le résiduel contient des arcs avec des coûts négatifs car à chaque arc original de coût c correspond un arc inverse de coût $-c$, et dès qu'on pousse du flot sur l'original, son inverse devient utilisable. On utilise donc Bellman-Ford qui sait gérer les coûts négatifs, plutôt que Dijkstra qui ne les supporte pas.

Le reste de l'algorithme est identique à Ford-Fulkerson : on trouve un chemin, on calcule la capacité résiduelle minimum dessus, on pousse le flot, et on recommence jusqu'à ce qu'il n'y ait plus de chemin. La seule différence c'est qu'on accumule aussi le coût total au fur et à mesure, en multipliant le coût unitaire du chemin par le delta poussé.

Le coût minimal est garanti car à chaque itération on choisit le chemin le moins cher disponible, donc on pousse d'abord les unités sur les chemins les moins coûteux, puis sur les plus coûteux quand les premiers sont saturés.

3.3 Min cost flow avec Dijkstra et renormalisation

Bellman-Ford fonctionne mais il est plus lent que Dijkstra, donc on aimerait pouvoir utiliser Dijkstra à la place. Le problème c'est qu'on a toujours des coûts négatifs dans le résiduel, et Dijkstra ne sait pas les gérer. La solution est de transformer les coûts pour les rendre tous positifs, sans changer le chemin optimal.

Pour ça, on associe à chaque sommet v un "potentiel" $h(v)$, et on remplace le coût de chaque arc $u \rightarrow v$ par son coût réduit :

$$c'(u \rightarrow v) = c(u \rightarrow v) + h(u) - h(v)$$

Si on choisit bien les potentiels, deux choses se passent : tous les coûts réduits deviennent positifs ou nuls, et le chemin de coût réduit minimum entre la source et le puits reste le même que le chemin de coût vrai minimum. Du coup on peut faire tourner Dijkstra.

Le bon choix de potentiels est de prendre $h(v)$ égal à la distance de la source à v calculée à l'itération précédente. Au tout premier tour on n'a rien, donc on initialise tous les potentiels à zéro, ce qui marche tant que tous les coûts originaux sont positifs ou nuls. Après chaque exécution de Dijkstra, on met à jour les potentiels en ajoutant les distances qu'on vient de calculer.

Pour le coût total qu'on accumule, on utilise les vrais coûts des arcs et pas les coûts réduits. La renormalisation permet juste de manipuler le graphe résiduel, et n'est donc pas utilisée pour le résultat.

3.4 Détection de cycles négatifs

Un cycle négatif dans le résiduel veut dire qu'on peut tourner en rond en réduisant le coût total, ce qui signifie que le flot courant n'est pas optimal. La théorie nous dit qu'avec les algorithmes par chemins augmentants successifs, le résiduel ne peut jamais contenir de cycle négatif, donc cet algorithme sert surtout à vérifier : on le lance après le min cost flow pour confirmer que le flot calculé est bien optimal.

L'idée est de réutiliser Bellman-Ford. Comme un plus court chemin ne peut pas avoir plus de $N - 1$ arcs, $N - 1$ passes suffisent à stabiliser toutes les distances dans un graphe sans cycle

négatif. On fait donc une passe supplémentaire après les $N - 1$ habituelles : si une distance peut encore être réduite, c'est qu'il existe un cycle négatif.

Une fois qu'on a détecté un sommet dont la distance change à la N -ième passe, on n'est pas sûr qu'il appartienne au cycle, il peut être avant. On remonte donc N fois la liste des arcs précédents, ce qui garantit qu'on est entré dans le cycle, puis on suit les sommets précédents jusqu'à revenir au sommet de départ pour reconstruire le cycle complet.

4 Tests et résultats

Le `Main` contient un test par algorithme, chacun construit son propre réseau et affiche le résultat via le `toString` de `ResultatFlotMax`.

Pour Ford-Fulkerson, on utilise un réseau classique à 6 sommets et 9 arcs sans coûts. Le résultat est un flot maximum de 23, et la coupe minimale a une capacité totale de 23, ce qui est cohérent avec le théorème max-flow / min-cut.

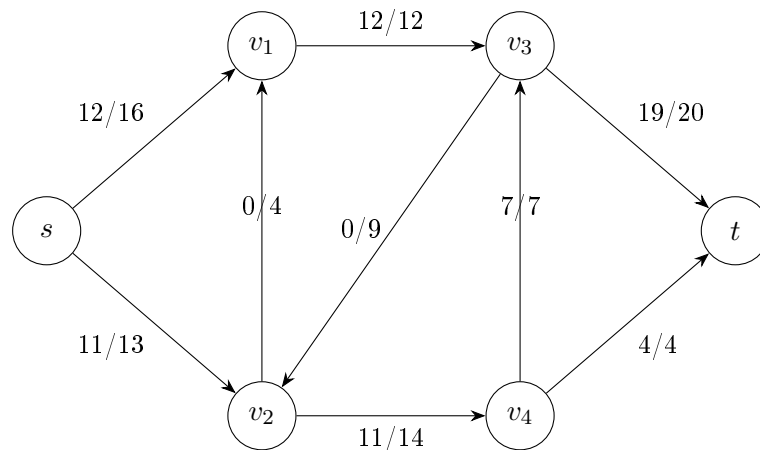


FIGURE 1 – Flot maximum sur le réseau de Ford-Fulkerson (flot/capacité, total = 23)

Pour les deux variantes du min cost flow, on utilise le même réseau à 4 sommets avec des coûts sur les arcs, ce qui permet de vérifier que Bellman-Ford et Dijkstra renormalisé donnent exactement le même résultat : un flot de 5 et un coût total de 21, avec les mêmes chemins découverts dans le même ordre..

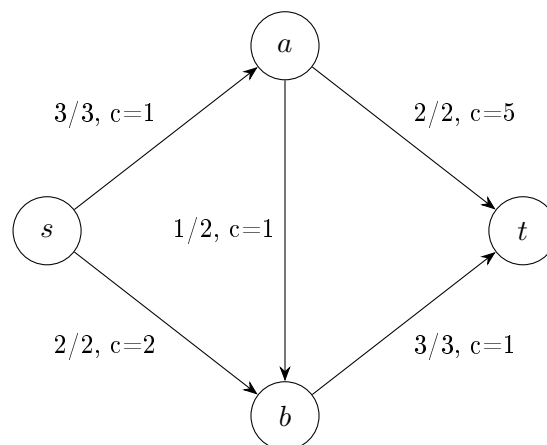


FIGURE 2 – Flot optimal sur le réseau de min cost flow (flot/capacité, coût ; total = 5, coût = 21)

Pour la détection de cycles négatifs, on la lance après le min cost flow sur le même réseau. Et l'algorithme ne trouve aucun cycle négatif, ce qui confirme que le flot calculé est optimal.

5 Conclusion

Les quatre algorithmes partagent la même structure de données et les mêmes utilitaires de parcours, ils ne diffèrent que par la méthode de recherche du chemin augmentant. C'est le choix de représenter le graphe résiduel par des paires d'arcs liés via un pointeur qui rend cette uniformité possible.